Section 1 Specification Table

Base model The base model is a model of the child hand game chopsticks using optimal play

Extension assumptions Introducing randomness and strategy, players will use strategy profiles rather than pure strategies to allow for the use of a markov chain symmetric states are counted separately so that the probability of winning can be calculated accurately. If there are multiple ways of winning the first one is chosen as the "optimal"

Technique 1: Game Theory Using backward induction and minimax to find the optimal moves for desired outcomes. Technique 2: Markov Chains The game is modelled as an absorbing markov chain with each state representing a game position and absorbing states representing wins/losses. Technique 3: Monte-Carlo Simulation Simulating the game to get empirical results

Question 1: At any given point in a game which player has the advantage with perfect play?
Question 2: What is the probability of victory from any state given specific strategies (random and epsilon greedy play).
Question 3: Are there steady states (i.e. draws) where no player can gain an advantage.\

Section 2 Introduction

The problem is the game of chopsticks [1] which is a finite, zero sum, 2 player strategy game. It is a combinatorial game but has few enough game states to be completely enumerated and solved. The base model assumes perfect play from both players. Perfect play from both sides in a childrens game is unrealistic and worse, uninteresting as it was found to always force a draw. The extension relaxes this to allow for epsilon-greedy play by introducing a chance that a player plays a random move rather than the optimal. Two applications of this extended model are examined. The first where Player 1 plays e-greedy against a Player 2 who plays randomely which isolates a skill asymetry. The second where both players play e-greedy strategies which models similar skill levels.

The first technique (MiniMaxing with backward induction) is used to solve the base model, finding optimal moves for each position. The second technique models a move as a set of probabilities which satisfy the markov property. Markov chain analysis is then used to discover how error rates and strategies change game dynamics. The third technique uses monte-carlo methodology to simulate games under each set of strategies, this is compared with the results of the second technique to verify our results.

[1] https://www.wikihow.com/Play-Chopsticks Note for simplicity "splitting" (transferring of fingers between hands) will not be allowed but, to allow for more game options hands

are added with mod-5 rollover (attacking a 4 with a 2 results in a 1) and are only eliminated at 0.

Section 3 Model description

Note shorthand player 1 left hand = P1L
player 1 right hand = P1R
player 2 left hand = P2L
player 2 right hand = P2R
Whose turn it is = T (1 for p1 2 for p2)

3.1 Rules and representation

Each state is a tuple of 5 values (P1L, P1R, P2L, P2R, T) with the start state being (1,1,1,1,1) and other states found via BFS. Terminal states are defined any state where one player has both hands = 0.

Moves are transitions between states, P1s possible moves are (P1L->P2L, P1L->P2R, P1R->P2L, P1R->P2R). The next state is the same as the previous except that the hand that was "attacked" has new value (attacking hand + attacked hand modulo 5). If a hands value is 0 it cannot be used to attack nor be targeted. Once both a players hands are 0, they are the loser and the opponent the winner. The game allows for draws where players moves ultimately loop back on themselves infinitely.

3.2 Base Model

Backward induction is performed via a retrograde algorithm where all terminal "Win states" are assigned a value (+1 for P1 win, -1 for P2 win). Values are then propagated backwards using MiniMax rules where values are maximised on P1 turn (I.e. if it's P1s turn and they can move to a winning/+1 state for them they will move to that state) and minimised for P2. Once all terminal states have been propagated (Measured by checking if any values have changed in a particular iteration), we determine that all other values do not lead to a win for either player and are assigned as draws (0 value). The base model is discrete, deterministic, finite, and solved by fixed point iteration.

3.3 Extended Model Absorbing markov chain

We define our two sets of transition rules. The first, epsilon greedy for both, where each there is a 1-e chance of picking the optimal move and an e chance of picking a random move. The second P2 random where P1 plays epsilon greedy and player 2 always plays random moves. Absorbing states (R) are determined as terminal win states and draw states, all other states are then transient states (Q). The model is solved analytically with Expected game length = N = (I-Q)^-1, and absorbtion probabilities = B = NR Each state is assigned an "outcome" value between -1 (100% chance player 2 wins from this position) and 1 (100% chance player 1 wins from this position)

3.4 Extended Model verification via Monte-Carlo.

We simulate 20,000 games for each set of epsilons varying from 0-1, under the two sets of transition rules, with each game going to a maximum of 5,000 steps. To accurately find the expected game length, once games enter draw loops they are declared as draws without going the full 5,000 steps. Win/draw probabilities are calculated via the total wins/draws divided by the number of simulations. The extended model is discrete, stochastic, linear and analytical.

3.5 Assumptions

- splitting is not allowed
- hands roll over modulo 5
- Hands are eliminated when they are equal to exactly 0
- When a state has multiple equal moves under minimax, the first one is chosen as optimal.
- Player skill levels and strategies remain the same throughout the game
- Symetric states are treated as equivalent when presented in graphs but are kept seperate in underlying graph representations.

Section 4 List of algorithms and methods

BFS - Used to find all reachable states, many states are not reachable which would make our analysis more computationally expensive without being more useful.

MiniMax with backward induction - Used to find the optimal strategy and the outcome of said strategy (+1 -1 or 0) for every reachable state.

SCC detection - Used to detect forced draw cycles where players cannot leave a loop under any policy. And used to identify policy-specific loops where players optimal policy leads to a forced draw.

Absorving Markov chain - Used to computing the exact win, loss, draw and expected lengths using varying levels of error and randomness.

Monte Carlo simulation - Used to empirically validate the markov chain results by simulating games

Section 5.0 Results

5.1 Base model state enumeration.

The base model enumeration via BFS found that out of a total of (5*5*5*2 = 1250) possible states, 1163 were reachable with 48 of those being terminal. However these include equivalent states. E.g. (1,0,1,0,1) == (1,0,0,1,1) == (0,1,0,1) == (1,0,1,0). Taking this into

account we get table 1, where Forced draw states are found as Strongly Connected components that never lead to a terminal state.

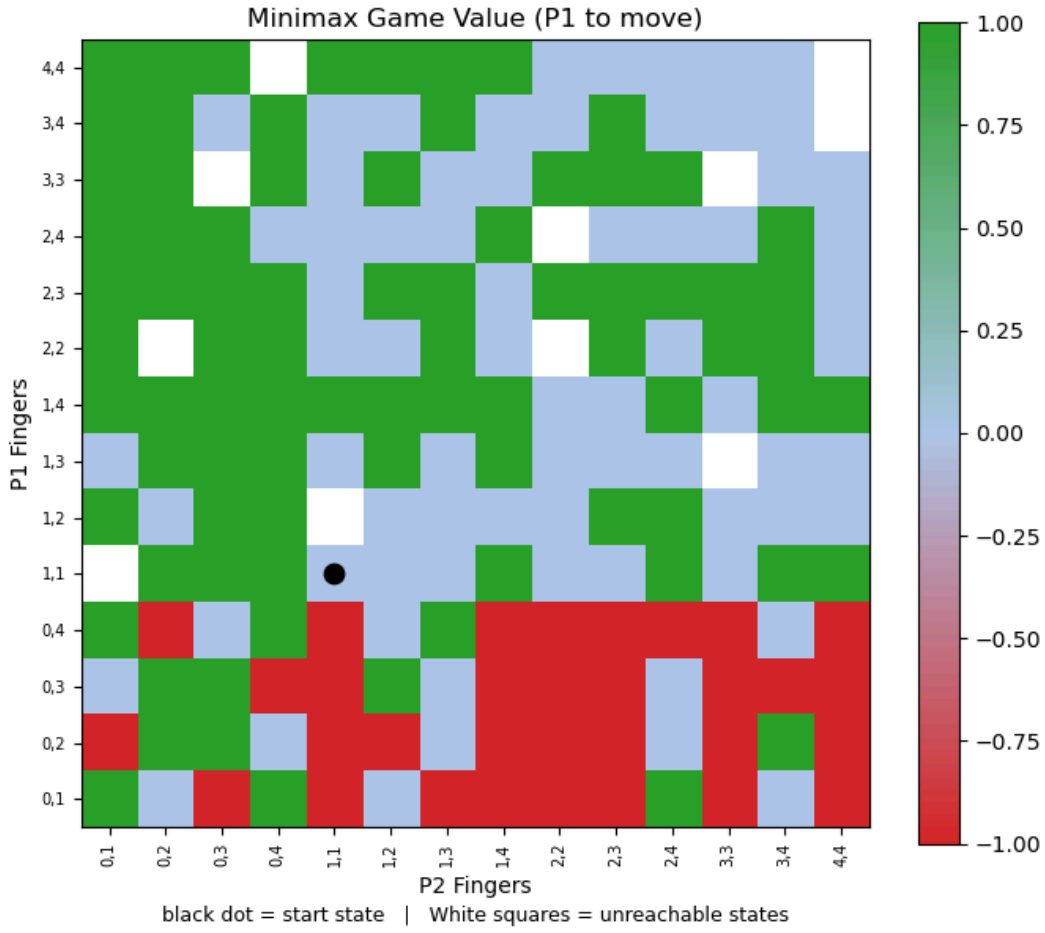
Table 1 STATE SPACE ENUMERATION (BFS) Total reachable : 398 Transient : 370 Terminal : 28 (P1 wins: 14, P2 wins: 14) Forced draws : 8

If both players remain in a repeating loop (which is a strongly connected component in the markov chain) then there are 2 possibilities.

1. the loop cannot be exited regardless of the play For example (1,0,2,0,1) -> (1,0,3,0,2) -> (4,0,3,0,1) -> (4,0,2,0,2) -> (1,0,2,0,1) which loops back
2. The loop cannot be exited with perfect policy For example, imagine a policy where each player only attacks the others left hand resulting in (1,1,2,1,1) -> (1,1,3,1,2) -> (4,1,3,1,1) -> (4,1,2,1,2) -> (1,1,2,1,1) which loops back Note this is the same example as 1 however in this case it is the players policy preventing them from leaving the loop rather than the rules of the game.

5.2 Base model analysis

Minimax analysis of the base model answers the first modelling question. Figure 1 shows that from the start state (black dot) neither player has an advantage and the game will result in a draw. The 28 win states (which would be row/column 0,0) are not shown. Across all states the player whose turn it has the advantage 96/225 possible states for a turn resulting in a P1 win and just 46/225 resulting in a p2 win. 71/225 states result in a draw and the remaining 12 are unreachable.

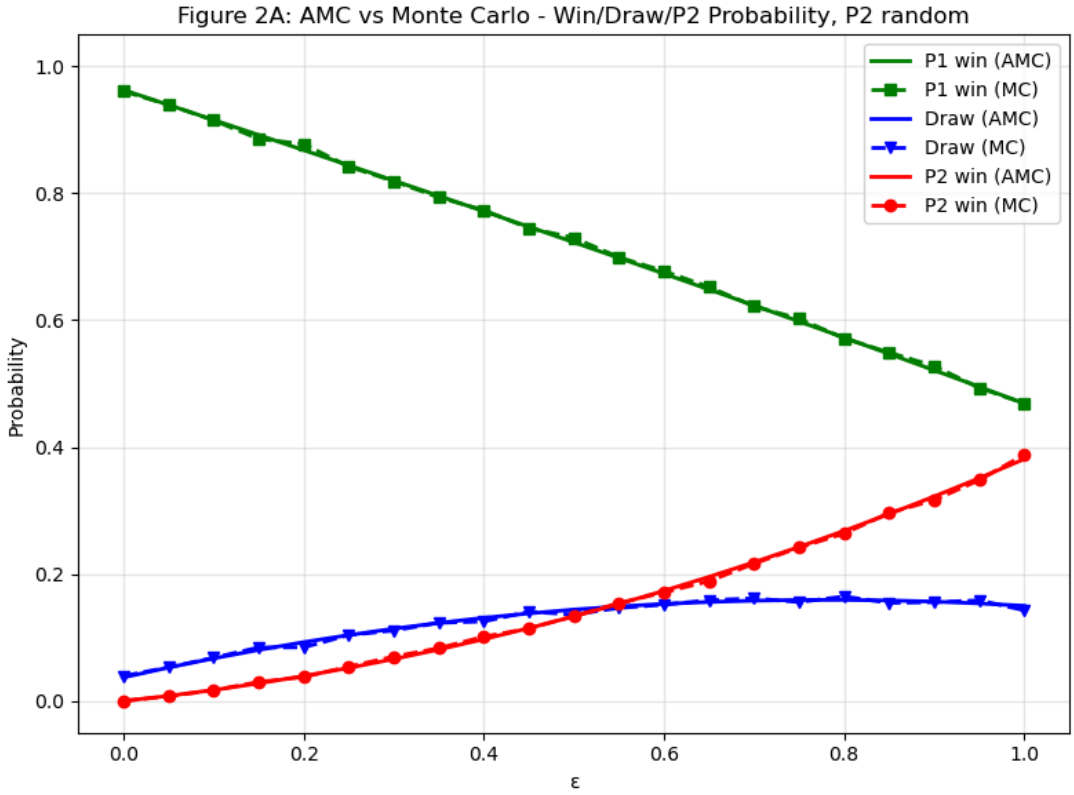


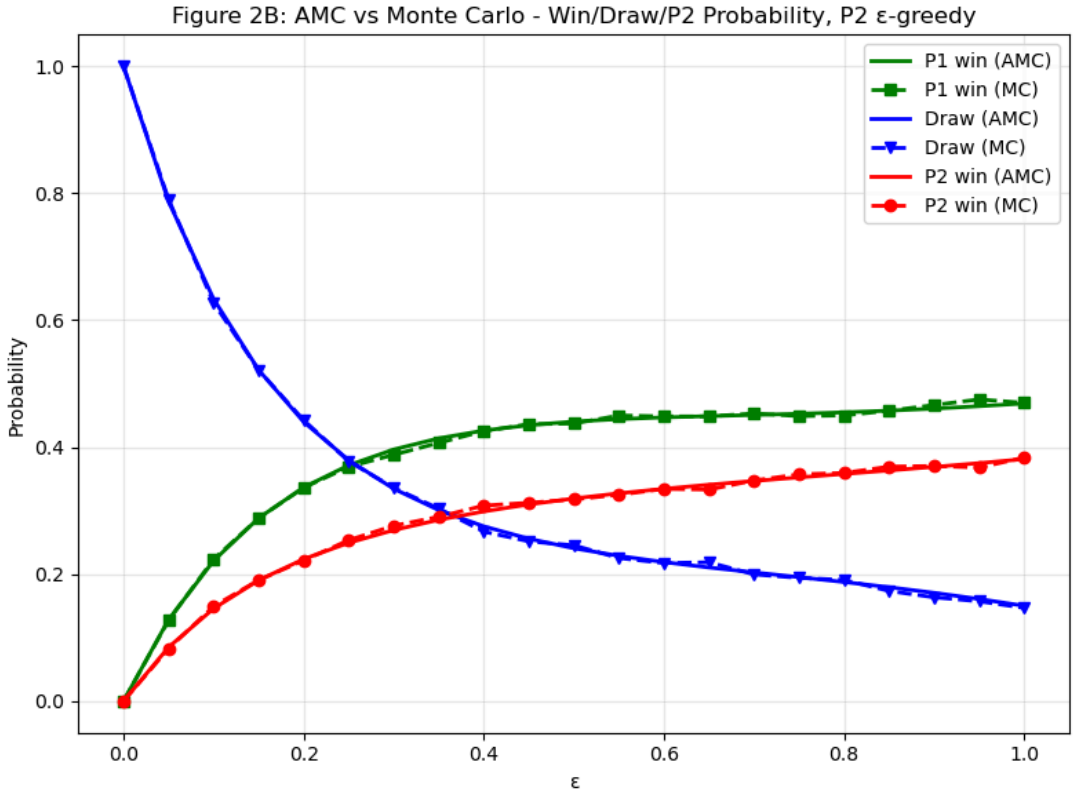
5.3 Extension analysis

Figure 2A shows P1 with perfect play vs P2 with random play across varying epsilon values. As could be expected, perfect play results in P1 winning the overwhelming majority (96.29%) of the time, with P2s random moves allowing it to draw occasionally (3.71%). Agreeing with Figure 1, Figure 2A shows P1 can never lose under perfect play.

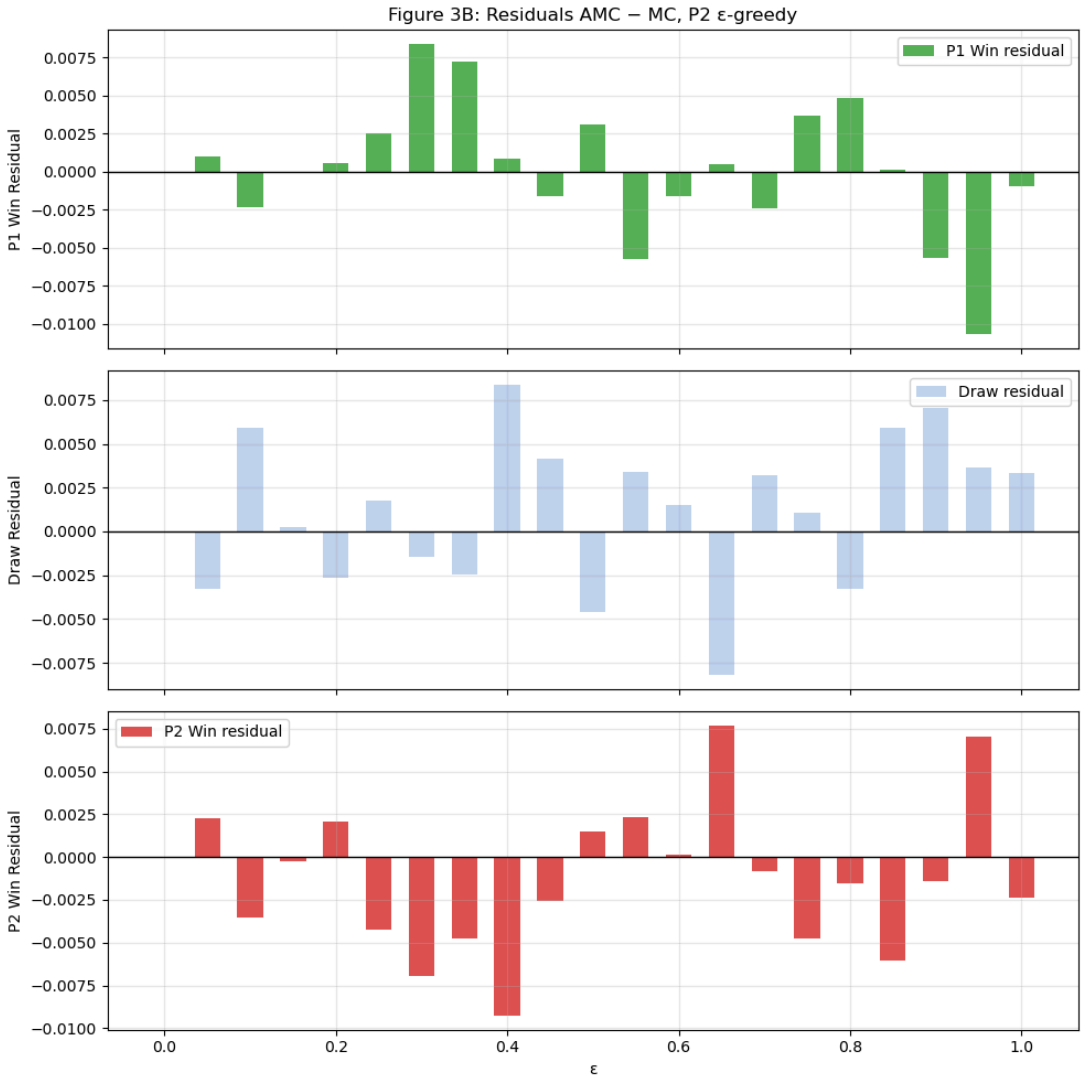
As epsilon increases, P1 tends to make more mistakes, allowing P2 to gain more draws and even some wins meaning their overall win rates converge. As epsilon reaches 1, P1 and P2 play with completely random moves. Still, P1 has an advantage in going first, winning 46.89% of the time compared with P2 winning 38.12% of the time.

Figure 2B shows both players with epsilon greedy play At epsilon = 0 (i.e. perfect play), no player can gain an advantage, however as epsilon increases, players make mistakes allowing their opponents to win. As with Figure 2A, Player 1 holds an advantage by going first. These results help answer question 2 by showing that with imperfect play, the player with the advantage is always the one that goes first regardless of error rates and imperfect play. They also help answer question 3, where lower error rate allows players to gain equal footing with their opponent, but even completely random play has a chance of obtaining a draw against perfect play





Figures 3A and 3B show the residual graphs comparing the results by modeling with absorbing markov chains and with Monte-Carlo simulation. The residuals are evenly weighted about the x axis and has a maximum error of around 1% showing extremely similar results between the Analytical and Empirical methods.


Figure 3B: Residuals AMC − MC, P2 ε-greedy
P1 Win residual
P1 Win Residual
Draw residual
Draw Residual
P2 Win residual
P2 Win Residual
0.0075
0.0050
0.0025
0.0000
−0.0025
−0.0050
−0.0075
−0.0100
0.0
0.2
0.4
0.6
0.8
1.0
ε

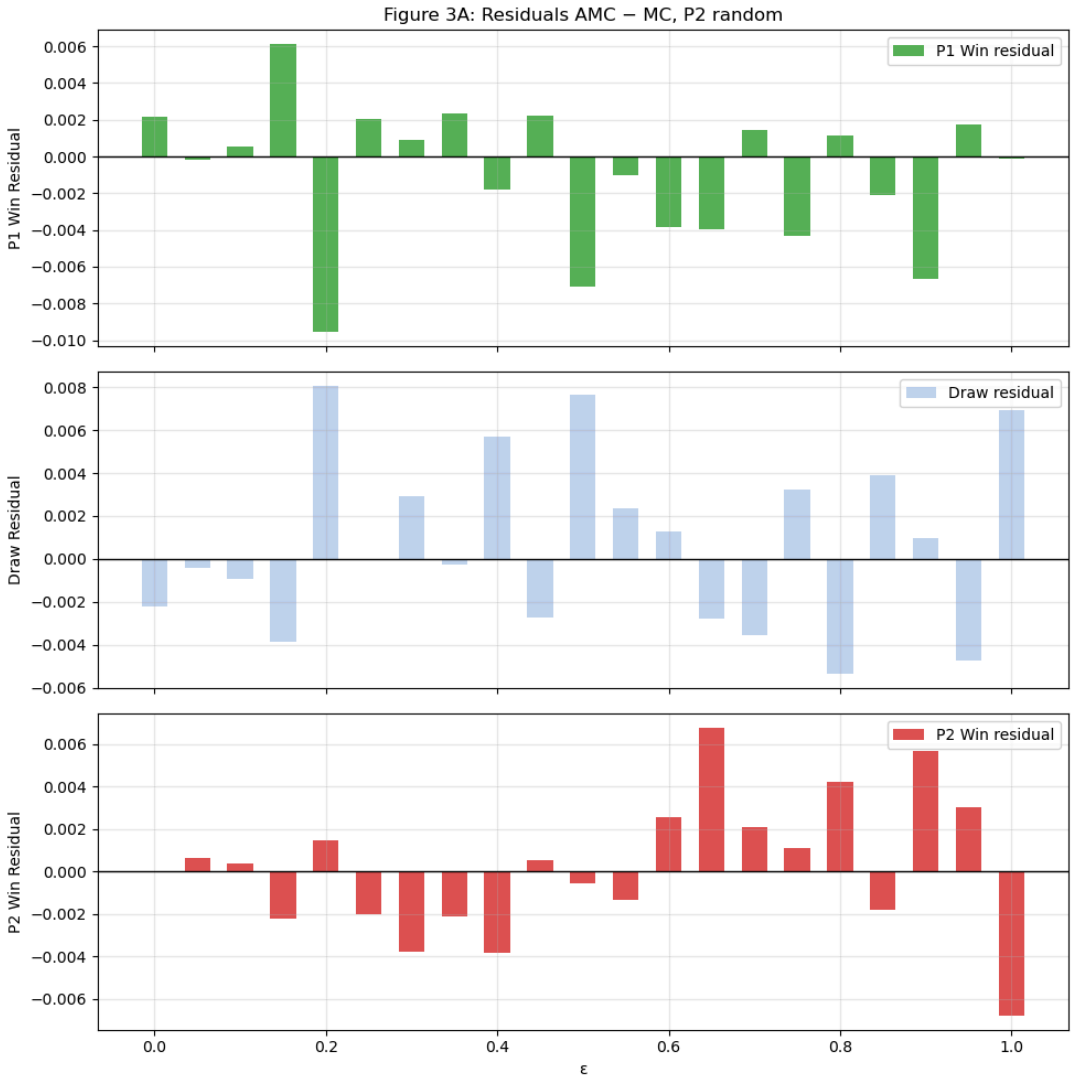


Figure 4A shows expected game length when P1 plays epsilon greedily and P2 plays random. As epsilon increases P1 plays more errors which results in a less optimal game extending the total game time. Figure 4B however shows both players playing epsilon greedily The game starts at a fixed length of 16 where both players play optimally leading to a draw. Adding a small amount of randomness (epsilon = 0.05) means a mistake or 2 sneak into players strategy, this leads to a longer, non-deterministic game which appears as a small spike in the graph. However as the number of mistakes increase, players are more easily able to take advantage of errors to move into winning and drawing states.

Figure 4A: Expected Game Length vs ε, P2 plays random

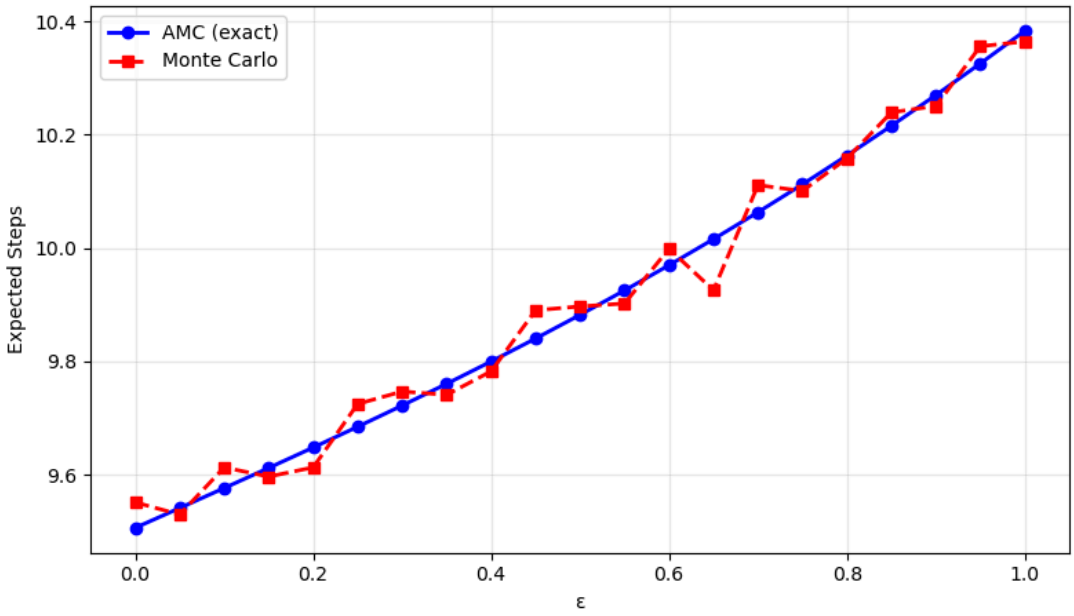
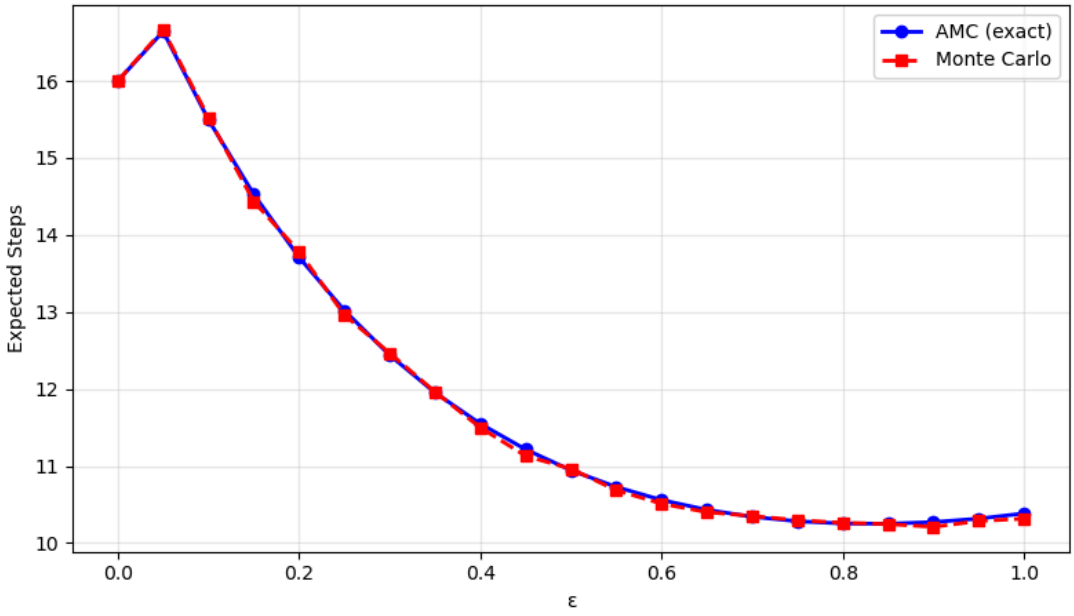


Figure 4B: Expected Game Length vs ε, P2 plays ε-greedy



Finally to answer all three questions Figure 5A shows win probabilities on player 1s turn from any position for varying epsilons against an entirely random opponent. It shows that while some positions guarentee a loss for Player 1, overall using perfect strategy and going first gives a major advantage. Figure 5B shows win probabilities against another player playing epsilon random. Here player 1 has a much more difficult time consistently winning, but still maintains an advantage on average.

Figure 5A: Win probability heatmap for various ε, P2 plays random

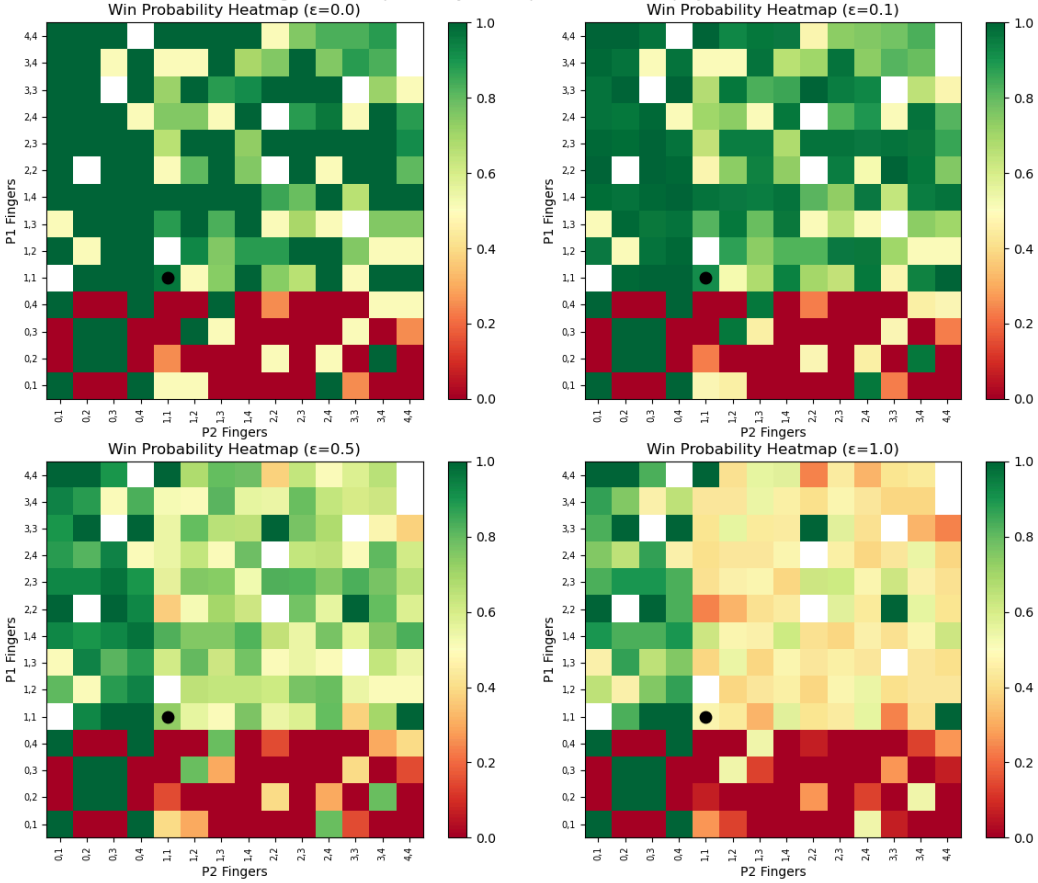
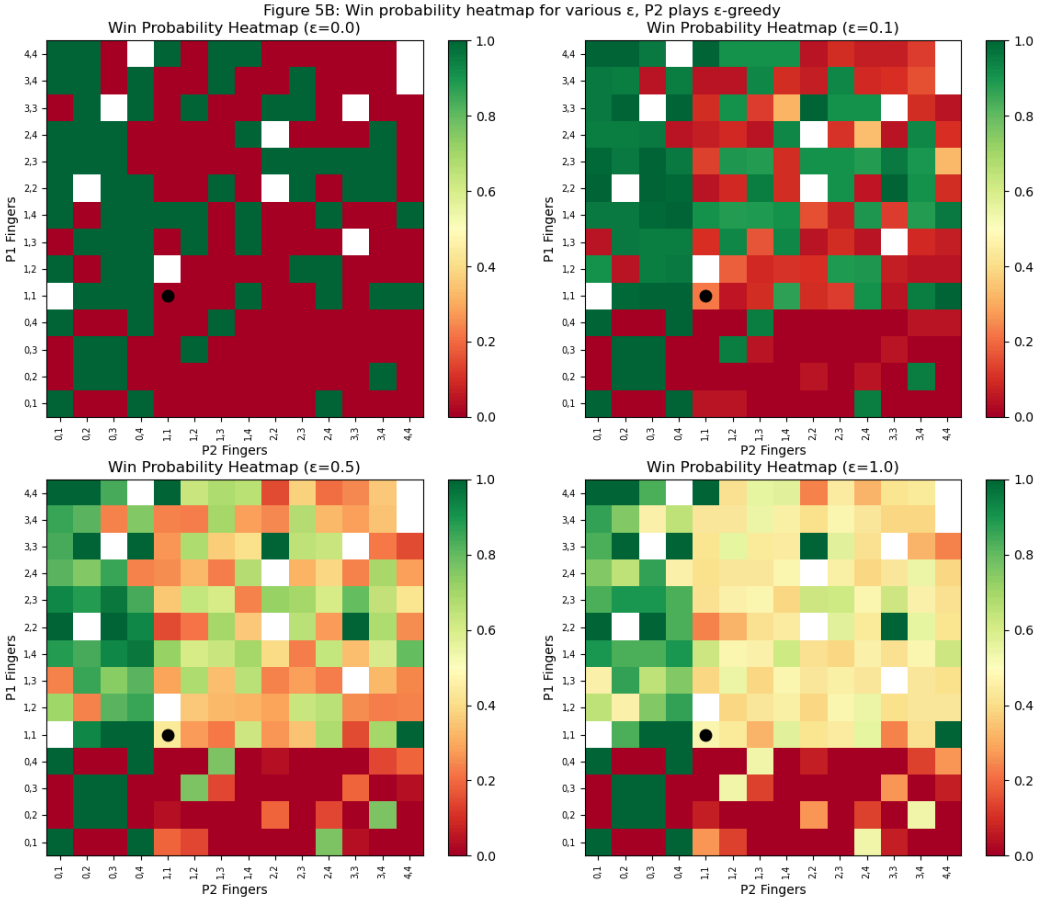


Figure 5B: Win probability heatmap for various ε, P2 plays ε-greedy



6 Conclusion

In conclusion the model chosen for the game of chopsticks under the assumptions stated show

1. At most given points in a game player one the advantage with perfect play\
2. Epsilon Greedy provides higher probability of victory from any state, with lower epsilons giving better results.
3. There are two kinds of steady states (draws) which are more common when players play with better strategies